

Douglass Brown

Arlecia Tyus

Security Policies & Procedures

May 24, 2026

Secure Code Review: Flask Login Application

Security Policies and Procedures Assignment

Overview

This review analyzes a small Flask login application that accepts credentials through URL parameters, builds a simulated SQL query with user input, and renders the result back to the browser. Although the app is a class demo, the patterns in the code represent common real-world web application risks.

Analyze: Vulnerabilities Identified

SQL Injection Pattern: The application constructs a SQL-like query using direct string interpolation. If this pattern were connected to a real database, an attacker could alter the query logic and potentially bypass authentication. **Reflected XSS / Unsafe Rendering:** The app uses `render_template_string()` with string concatenation and includes user-controlled input in the returned HTML. This can allow injected HTML or JavaScript to execute in a user browser. **Credentials in URL Parameters:** The app accepts username and password through a GET request. This exposes credentials in browser history, logs, screenshots, proxy records, and referrer headers. **Hardcoded Plaintext Passwords:** Passwords are stored directly in the source code. If the code is leaked, valid credentials are exposed immediately. **Debug Mode Enabled:** Running Flask with `debug=True` can expose sensitive stack traces and internal application details. Debug mode should not be enabled outside development.

Exploit: Example Attack Paths

A SQL injection attempt could use a username such as:

```
/login?username=admin'--&password=anything
```

In a real SQL-backed application, this could comment out the password check and authenticate as admin. A reflected XSS attempt could use a username such as:

```
/login?username=<script>alert('XSS')</script>&password=test
```

Because the app returns user input inside rendered HTML, the browser may interpret the injected script. A normal login URL also exposes credentials, for example:

```
/login?username=admin&password=adminpassword
```

Propose a Fix: Modified Code

```
from flask import Flask, request, render_template_string
from werkzeug.security import generate_password_hash, check_password_hash
import os

app = Flask(__name__)

USERS = {
    "admin": generate_password_hash("adminpassword"),
    "user": generate_password_hash("userpassword")
}

LOGIN_PAGE = """
<!DOCTYPE html>
<html>
<head><title>Secure Login Demo</title></head>
<body>
    <h1>Secure Login</h1>
    {% if message %}<p>{{ message }}</p>{% endif %}
    <form method="POST" action="/login">
        <label>Username:</label>
        <input type="text" name="username" required><br><br>
        <label>Password:</label>
        <input type="password" name="password" required><br><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>
"""

@app.route("/")
def home():
    return render_template_string(LOGIN_PAGE, message=None)

@app.route("/login", methods=["POST"])
def login():
    username = request.form.get("username", "").strip()
    password = request.form.get("password", "")

    response = "Invalid username or password."
    if username in USERS and check_password_hash(USERS[username], password):
        response = f"Successfully logged in as {username}."

    return render_template_string(LOGIN_PAGE, message=response)

if __name__ == "__main__":
    debug_mode = os.environ.get("FLASK_DEBUG", "false").lower() == "true"
    app.run(debug=debug_mode)
```

Security Implications

The original code weakens confidentiality, integrity, and access control. SQL injection could allow unauthorized access or data manipulation in a real database-backed version. Reflected XSS could let an attacker run malicious code in a victim browser, leading to session theft, phishing, or unauthorized actions. Exposing credentials in URLs also creates unnecessary leakage through logs and history. Hardcoded plaintext passwords and debug mode further increase the risk of credential compromise and information disclosure.

Reflection and Recommendations

- Use POST requests for login forms instead of GET requests.
- Never include passwords in URLs.
- Use parameterized queries for all database access.
- Store hashed passwords, not plaintext passwords.
- Avoid rendering raw user input directly into HTML.
- Disable debug mode outside development.
- Use generic login errors to avoid username enumeration.
- Add HTTPS, CSRF protection, rate limiting, secure session management, and logging for a production system.

Conclusion

The corrected version improves the application by reducing injection risk, protecting credentials, avoiding unsafe output rendering, and removing insecure development behavior. The review demonstrates how small design decisions in authentication code can have major effects on an application security posture.